

ECE 498 BH — LLM Reasoning for Engineering

Final Project Guidelines

Due 11:59 pm, 05/19/2026

Objective

In Assignments 1–5 you worked with a fixed problem suite, hand-crafted verifiers, and a single pipeline. That setup was useful because it let us isolate one variable at a time and measure it cleanly.

The final project asks you to move one step closer to realistic engineering use. In real engineering work, the problem is rarely handed to you as a clean benchmark. You often have to decide what question is worth asking, what inputs are realistic, what tools or context the model should use, and how to tell whether the output is actually correct or useful.

Your project should therefore pick a direction that matters in your field and build a small, honest study around it. What “matters” depends on the field: a realistic signal-processing question is not the same as a realistic digital-hardware question. **You choose the domain and the question.** The deliverable is a 4–8 page report plus working code that demonstrates the result.

What Every Project Must Contain

Five elements, regardless of direction:

1. **A specific question.** One paragraph: what are you studying and why? Specific enough that someone reading it can imagine being wrong about the answer.
2. **A prior hypothesis or evaluation question.** Before any substantial runs, write your prediction or primary evaluation question in `predictions.md` (dated). It must be specific enough that the result could surprise you. “Tool X reduces numerical errors from N to roughly M ” qualifies. “Most failures will come from missing physical constraints rather than syntax errors” qualifies. “Tool X helps” does not.
3. **A controlled comparison or controlled characterization.** Most projects should include at least two conditions differing on one axis. If your project is a benchmark or

failure analysis, the controlled object may instead be a curated set of problem features. In either case, the comparison must be apples-to-apples; if it isn't, say so.

4. **Validation that distinguishes *execution* from *domain* success.** Code that runs is not a result. Your project must answer: *how do we know the output is correct or useful?* A verifier in the HW4 sense is the cleanest path. If your domain doesn't admit a clean verifier, describe and justify your alternative (LLM-as-judge with held-out reference, simulation-based scoring, expert inspection of a labeled sample). The alternative must be defensible against the obvious objection. This is the component we weight most heavily.
5. **Honest discussion.** What does your result depend on (domain, model, distribution)? What is the strongest argument against your conclusion? Engage it.

Most projects should involve roughly 5–20 carefully chosen cases, or a well-justified subset of an existing benchmark used in your chosen field.

Reusing HW Infrastructure

Reuse the HW4 verifier interface (Pydantic + pass/details/reason), the HW5 memory store, and the per-trial JSON log format wherever they fit. Build new pieces whenever needed.

Suggested Process

This is a suggested process. Adjust it to your project; you do not need to follow it exactly.

First, pick the question. Write `predictions.md`. Sketch the experimental design. Then get the suite, the verifier or alternative validation method, and the pipeline ready. **Pilot** on a small slice before scaling. This catches bugs early. You are allowed to use LLMs to help you code, but you need to understand the LLM-generated codes yourself.

Finally, run the full study, read the failures, classify them, compare the results to your prediction or evaluation question, and write up what you found. Report your findings honestly, even if the proposed technique does not work in the end.

What We Grade On

Roughly, in decreasing weight:

- **Validation.** Most important. A polished demo without validation does not pass.
- **Question and comparison.** Is the question well-posed? Does the comparison or characterization isolate what you claim to measure?
- **Honesty.** Negative or ambiguous results are fine if reported clearly. Reporting a result you didn't actually measure is not.

- **Reproducibility.** Can another competent person clone the repo, run the demo, and see evidence the output is correct or useful?

We are not grading on whether your hypothesis was right, pipeline complexity, problem-suite size, or domain novelty. A 5-page report with clear results beats a 10-page report with hedged ones.

Submission

- ☐ `report.pdf` (4–8 pages).
- ☐ `predictions.md` (dated, written before substantial runs).
- ☐ Source code, including the script that produces your results.
- ☐ Results artifact: the table, figure, or notebook cell that contains your main comparison.
- ☐ Per-trial logs (JSON or equivalent). Without these the results cannot be audited.
- ☐ `README.md`: environment, dependencies, how to run the demo, expected output, known failure modes.

API keys must not appear in any submitted file. Use environment variables.

The Standard

A final project succeeds if another technically competent person can clone the repository, run the demo, understand the pipeline, and see evidence that the output is correct or useful.